Foundation Modules (Broad Audience) A Series – Foundations & Architecture

DAY 1: Foundations of Agentic AI, Azure Infrastructure & State Machines

Theory Topics & Subtopics

- **Agentic AI Architecture:**
  - Chatbots vs. Autonomous Agents vs. State Machines.
  - System State Schemas: Defining a single source of truth using TypedDict.
- **Enterprise AI Infrastructure & Governance:**
  - Authentication, endpoints, and credentials handling.
  - Static token credential adapters for cloud AI services.
  - System telemetry: Implementing structured logging over basic console logs.
- **State Machines with LangGraph:**
  - Graph mechanics: StateGraph, Nodes, Edges, START, and END.
  - Conditional Logic: Dynamic routing functions based on state updates.

Practical Hands-on Labs Overview

- **Lab 1: Environment Setup & Enterprise Telemetry Configuration:** We start by setting up our Python development environment, installing core dependencies (langgraph, azure-ai-projects, azure-identity), and initializing structured logging to capture real-time execution logs.
- **Lab 2: Azure AI Endpoint Integration & Client Authentication:** Next, we configure authentication using a static token credential adapter to securely connect our application to the Azure AI model deployment.
- **Lab 3: Building the State Memory Schema:** We define a centralized state schema using Python's TypedDict to manage and pass variable states safely across different execution nodes.
- **Lab 4: Developing Execution Nodes & Dynamic Branching Rules:** We write functional nodes to process incoming data and build dynamic routing logic that directs execution flow based on state criteria.
- **Lab 5: Compiling and Invoking the LangGraph Engine:** Finally, we assemble the nodes, edges, and conditional routers into a StateGraph, compile the agent engine, and run test executions to verify state updates.

DAY 2: RAG Pipelines, Vector Search & Tool-Augmented Agents

Theory Topics & Subtopics

- **Unstructured Data Processing & Vector Storage:**
  - Document Chunking, Metadata, Unique IDs, and Vector Embeddings.
  - Vector Database Administration using Chroma DB.
- **Retrieval-Augmented Generation (RAG):**
  - Semantic Search: Querying vector collections using natural language text queries.
  - Evidence Grounding: Restricting model responses strictly to retrieved context.
- **Tool-Augmented State Graphs:**
  - Function Calling: Wrapping external system lookups and calculations as agent tools.
  - Extended State Schemas: Tracking tool inputs and context outputs across graph nodes.

Practical Hands-on Labs Overview

- **Lab 1: Vector Database Initialization & Document Ingestion:** We start by initializing a local persistent Chroma Vector DB, creating collections, and loading unstructured text chunks along with metadata.
- **Lab 2: Building External System Integration Tools:** We create functional tools for database lookups, balance calculations, and record creation that the agent can execute during runtime.
- **Lab 3: Implementing a Semantic Vector Search Node:** We build a search node that converts incoming queries into vector searches against Chroma DB to retrieve relevant document context.
- **Lab 4: Building Grounded Prompt Nodes:** We write AI processing nodes that enforce strict grounding, requiring the model to generate decisions based solely on retrieved document context.
- **Lab 5: Assembling the Multi-Node RAG Pipeline:** Finally, we connect our tool nodes, vector search nodes, and grounded reasoning nodes into a unified LangGraph workflow and run end-to-end tests.

DAY 3: Enterprise Governance, Security Guardrails & System Observability

Theory Topics & Subtopics

- **LLM Vulnerabilities & Security Vectors:**
  - Prompt Injection: System overrides, keyword hijacking, and instruction bypasses.
  - Data Leakage: Raw API dumps, sensitive token exposure, and PII leaks.
- **Enterprise Guardrail Controls:**
  - Input Sanitization: Filtering disallowed patterns and malicious instructions.
  - Output Redaction: Structural masking and sensitive data stripping gates.
  - Security State Execution Holds (REJECTED_SECURITY_HOLD).
- **System Observability & Auditing:**
  - Real-time state transition tracking across execution lifecycles.

Practical Hands-on Labs Overview

- **Lab 1: Constructing Input Guardrails (Anti-Prompt Injection Shield):** We start by writing input validation filters using pattern matching and keyword checking to detect and block prompt injection attempts before they reach the model.
- **Lab 2: Developing Output Sanitization & Masking Gates:** We build output filters that analyze response structures to catch sensitive internal parameters (like API keys or internal tokens) and convert them into clean response formats.
- **Lab 3: Implementing Real-Time Audit & Transition Logging:** We set up a state transition logger that records every status change as data passes through the security and processing nodes.
- **Lab 4: Building Secured Pipeline Processing Nodes:** We combine our security guardrails, business logic, and output sanitizers into a secured node pipeline that halts execution when security anomalies occur.
- **Lab 5: Executing Security Scenario Matrix Verification:** Finally, we stress-test the entire system against three real-world test cases: clean standard runs, malicious prompt injection attacks, and sensitive data exposure scenarios to verify our guardrails work as expected.

Accenture Batch 1: Automated Payment & Reconciliation System (Unified Capstone 01 & 02)

This comprehensive curriculum and project guide unifies Capstone 01 (Configurable Rule Engine) and Capstone 02 (Deduction Identification & Cash Application) into a single enterprise-grade solution using Agentic AI, LangGraph, and Microsoft Foundry.

1. Executive Summary & Core Business Problem

In modern enterprise accounting, receiving payments from corporate clients is rarely straightforward. Customers frequently pay lump sums covering dozens of invoices, withhold partial amounts due to damaged goods, or reference internal Purchase Orders rather than billing invoice numbers.

> **Key Business Challenge:** Manual reconciliation consumes hundreds of hours, increases cash posting errors, and leaves outstanding disputes unresolved. This project automates the end-to-end payment lifecycle using Agentic AI workflows.

The 3 Master Data Sources

- **Bank Statement:** Confirmed incoming monetary transfers (e.g., "$9,500 transferred from Acme Corp").
- **ERP Accounts Receivable (AR):** Billed amounts and open invoice records stored in the enterprise database.
- **Remittance Advice:** Contextual notes, PDFs, or emails sent by the customer detailing invoice breakdowns and deduction justifications.

The 6 Automated End States

Sequence	End State Description	System Status
1	Bank Transaction uploaded and not matched with Open AR/Remittance	Open
2	Bank Statement & Open AR/Remittance Value matched with variance	Partial Match

Sequence	End State Description	System Status
3	Bank Statement & Open AR/Remittance Value fully matched	Closed
4	No Invoice details in Bank Statement & No Remittance Advice	UAC (Un-applied Cash)
5	No information in Bank Statement	UIC (Un-Identified Cash)
6	User queried for more information / Low confidence edge case	Query

2. Priority Matching Rules & Practical Examples

The system evaluates incoming payments sequentially across configurable priority levels to find matching records in the ERP database.

Priority	Bank Statement Fields	ERP AR Open Fields	Remittance Fields	Match Type	Partial Match Action
1	Customer Name + PO Number + Amount	Customer Name + PO Number (lookup Invoice) + Amount	N/A	2-Way	Post unmatched variance to UAC

Priority	Bank Statement Fields	ERP AR Open Fields	Remittance Fields	Match Type	Partial Match Action
2	Customer Name + Delivery Number + Amount	Customer Name + Delivery Number (lookup Invoice) + Amount	N/A	2-Way	Post unmatched variance to UAC
3	Customer Name + Invoice Number + Invoice Date + Amount	Customer Name + Invoice Number + Invoice Date + Amount	N/A	2-Way	Post unmatched variance to UAC
4	Customer Name + Invoice Number + Amount	Customer Name + Invoice Number + Amount	N/A	2-Way	Post unmatched variance to UAC
5	Customer Bank Payment	Customer Name + PO + Invoice + Date + Amount	Customer Name + PO + Invoice + Date + Amount	3-Way	Post unmatched variance to UAC
6	Customer Bank Payment	Customer Name + PO + Invoice + Amount	Customer Name + PO + Invoice + Amount	3-Way	Post unmatched variance to UAC

Detailed Scenario Examples

Example A: Perfect Match (Priority 4 - Positive)

- **Bank Input:** Customer: Acme Corp | Invoice: INV-808 | Paid: $10,000
- **ERP Record:** Customer: Acme Corp | Invoice: INV-808 | Amount: $10,000
- **Outcome:** 2-Way Match Successful. **End State: CLOSED**

Example B: Short Payment & Deduction Capture (Partial Match)

- **Invoice Amount:** $10,000
- **Payment Received:** $9,500
- **Remittance Note:** "5 units damaged in transit ($500 balance withheld)."
- **Logic Execution:** Identifies $500 short payment -> Maps reason to Code **D03 (Damage)** -> Posts $9,500 to close paid portion -> Opens dispute item for $500.
- **Outcome: End State: PARTIAL MATCH**

Example C: Auto-Match via Tolerance

- **Invoice Amount:** $10,000 | **Payment Received:** $9,995
- **Tolerance Setting:** Differences <= $10 are auto-written off.
- **Outcome:** $5 difference written off automatically. **End State: CLOSED**

Example D: Un-applied & Un-identified Cash (Negative Cases)

- **Case 1 (UAC):** $5,000 received from "Wayne Enterprises" with no invoice or PO details. Customer identified, invoice unknown. **End State: UAC**
- **Case 2 (UIC):** Wire transfer of $2,000 received with completely blank metadata. Sender unknown. **End State: UIC**

3. Deduction Reason Mapping & Standard Codes

Reason Code	Deduction Category	Description / Trigger Scenario
D01	Pricing Issue	Discrepancy between contracted unit price and billed invoice price.

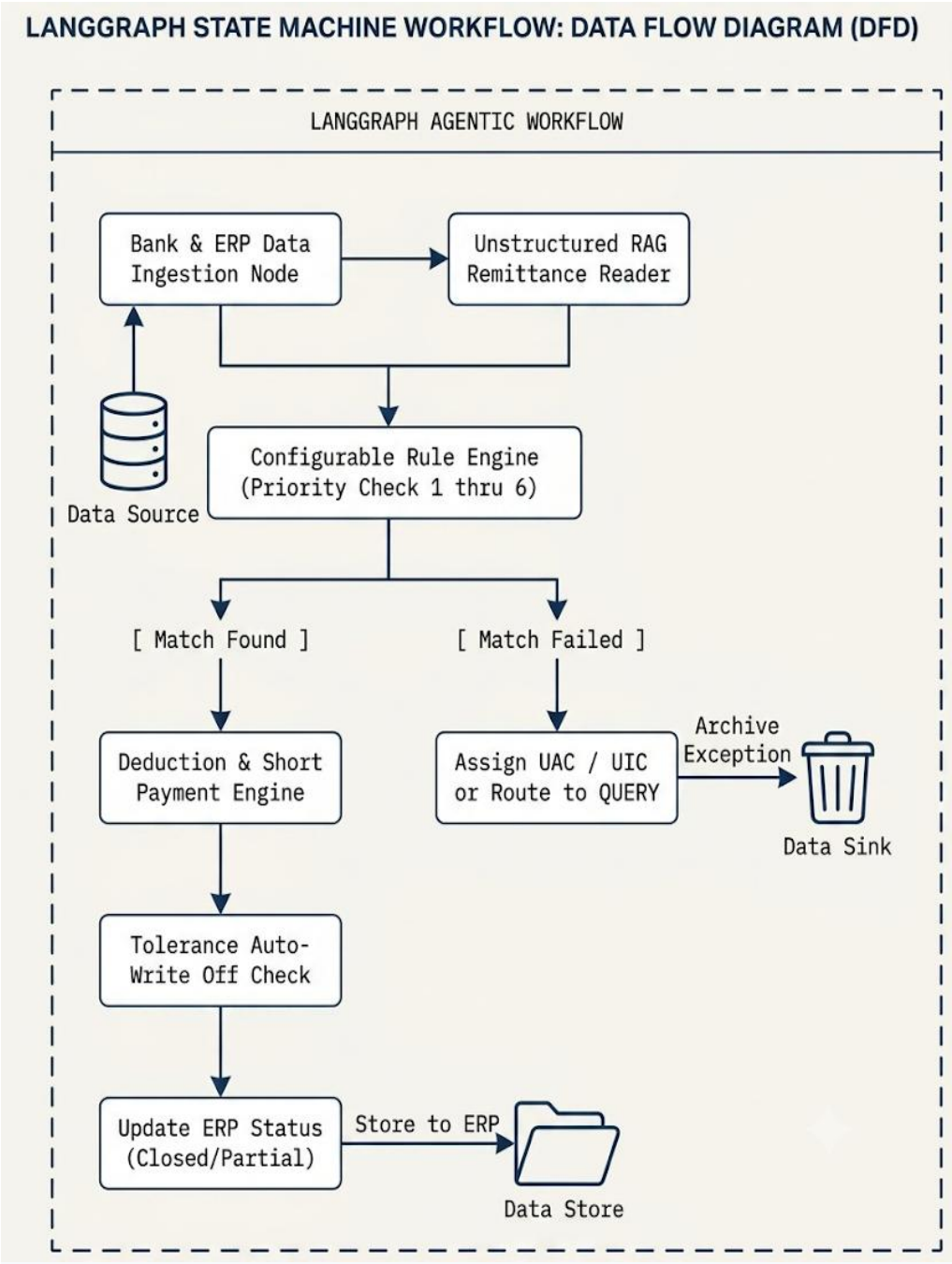
Reason Code	Deduction Category	Description / Trigger Scenario
D02	Freight Claim	Unauthorized shipping or handling charges billed to customer.
D03	Damage Claim	Goods received broken, corrupted, or unusable.
D04	Tax Difference	Exempt tax status claimed or state sales tax variance.
D05	Discount Taken	Early payment discount taken outside allowed payment terms.

4. Technical Architecture & Flow Diagrams

Technical Stack Integration Summary

- **LangGraph State Machine:** Controls execution flow, handles branching logic for 2-way and 3-way matches, and manages Human-In-The-Loop interruptions for the QUERY state.
- **Agentic RAG & Microsoft Foundry:** Parses unstructured PDF receipts, emails, and scanned remittance files, extracting unstructured deduction reasons and mapping them to standardized reason codes (D01–D05).
- **Model Context Protocol (MCP) Tools:** Provides structured, secure tool calling capabilities for AI agents to query underlying ERP SQL tables and read bank ledger files without direct database exposure.
- **Observability & Guardrails:** Ensures accurate monetary posting, prevents hallucinated match keys, and logs complete reasoning traces for financial auditing compliance.

LangGraph State Machine Workflow



5. End-to-End System Decision Logic Flowchart